PROJECT REQUIREMENT DOCUMENT

Website Scraper + Local LLM (Ollama) Knowledge-Base Chatbot

1. Project Overview

The goal of this project is to build an end-to-end JavaScript application that lets a user enter a website URL, scrapes the content of that site, converts the content into a searchable knowledge base, and exposes that knowledge base through a chatbot powered by a locally-hosted LLM (via Ollama). The user should be able to ask natural-language questions about the website and get accurate, context-grounded answers.

2. Objectives

• Understand, at a working level, what web scraping, embeddings, vector search, and RAG (Retrieval-Augmented Generation) each do — before writing code for them.

• Build a JavaScript-based web scraper that accepts a URL and crawls a small number of pages on that site.

• Clean and structure the scraped content into a usable knowledge base.

• Generate embeddings and store them so relevant content can be retrieved for a given question.

• Connect a locally-running Ollama LLM to answer questions using retrieved content.

• Build a simple web-based chatbot UI where a user can enter a URL and then chat about that site's content.

3. Pre-Work / Learning Primer (Days 1)

Before writing the main application, each intern should be able to explain the following in their own words. This isn't graded homework — it's the foundation that makes the rest of the project make sense instead of feeling like copy-pasting code.

• What an API call is, and how async/await works in JavaScript (if not already comfortable with this).

• What "scraping" means: fetching a page's HTML and pulling out the text you care about.

• What an embedding is: a way of turning text into a list of numbers that captures its meaning, so similar text ends up with similar numbers.

• What a vector/similarity search does: given a question's embedding, find the stored chunks whose embeddings are most similar.

- What RAG means: instead of asking the LLM a question cold, you first retrieve relevant text and hand it to the LLM as context, so it answers using that content instead of guessing.

- How to install Ollama, pull a model (e.g. `ollama pull llama3`), and send it a test prompt from the terminal.